

# Analysis of Stock Market Performance

## After VIX Spikes

*Final Project Report*

|                   |                                           |
|-------------------|-------------------------------------------|
| <b>Student</b>    | Chase Pellegrom                           |
| <b>Instructor</b> | Dr. Christine Liebe                       |
| <b>Course</b>     | CSCI128: Introduction to Computer Science |
| <b>Date</b>       | August 15, 2025                           |

## 1. Research Question and Dataset Description

This project investigates how stock prices respond after a large increase in the VIX (Cboe Volatility Index), commonly referred to as the market's "fear index" or "fear gauge"; it is a real-time market index that measures the market's 30-day expected volatility of the S&P 500 Index and acts as an indicator rather than a tradable asset. The central question being investigated is how the return of a stock, ETF, or index changes N days after the VIX spikes above 20 percent.

The analysis involves using historical daily closing price data for the VIX (^VIX) and a user-selected stock or ETF; in this report, the S&P 500 (SPY) will be used. Data was obtained through Yahoo Finance using the Python library `yfinance`. Each dataset contains two core fields: Date, the trading day, and Close, that day's closing price. For offline execution and submission, the project folder includes `SPY.csv` and `vix_data.csv` pre-uploaded to the code.

## 2. Program Files and Execution Instructions

The required submission files are `submission.py`, `SPY.csv`, `vix_data.csv`, `STDIN`, and `requirements.txt`. An additional utility file, `make_data_once.py`, was used during development to generate CSV files for other tickers but was not included in the final submission because it interfered with the autograder.

The utility script downloads approximately 20 years of daily price data, extracts the Date and Close columns, and writes the output to CSV files for both the VIX and a selected ticker.

## Script

```
import pandas as pd
import yfinance as yf

def save_csv(ticker, outname):
    df = yf.download(ticker, period="20y", interval="1d",
                     progress=False)
    if df.empty:
        raise SystemExit(f"Download failed for {ticker}")
    df = df.reset_index()[["Date", "Close"]]
    df.to_csv(outname, index=False)
    print(f"Wrote {outname} with {len(df)} rows")

save_csv("^VIX", "vix_data.csv")

# Change ticker here:
save_csv("SPY", "SPY.csv")
```

To run the program, navigate to the project folder in a terminal and execute python submission.py. The program will then prompt the user to enter a ticker symbol, the number of days after the spike to analyze, and the spike percentage threshold (the spike threshold must be greater than or equal to 20 percent, since smaller increases are not considered spikes in this context).

### 3. Testing Report

The project included three unit tests: test\_percent\_change(), test\_success\_rate\_basic(), and test\_detect\_vix\_spikes\_simple(). Together, these tests verify the core calculations and filtering logic used in the analysis.

- test\_percent\_change() checks a standard case in which a price rises from 100 to 110, corresponding to a 10 percent increase. This function also includes an edge case for division by zero.

- `test_success_rate_basic()` evaluates the `success_rate` calculation using a mix of positive, zero, and negative returns, ensuring that the resulting percentage is computed correctly.
- `test_detect_vix_spikes_simple()` uses a small synthetic dataset to verify that the spike-detection logic correctly identifies qualifying spikes.

Testing helped identify several early logic errors, including failing to guard against division by zero, forgetting to convert values to percentages, and accidentally including spikes that did not meet the specified threshold.

#### 4. Student vs. AI Function Analysis

A required component of the project was to submit an original code snippet to an AI model and ask it to simplify or improve the implementation, then compare the AI-generated version with the student-written version.

##### Original Version

```
def compute_returns(spike_dates, stock_data,
                    days_after):
    returns = []
    for date in spike_dates:
        # Filter through stock Series to all dates
        # on or after that target date
        start_price = stock_data[stock_data.index >= date]
        if start_price.empty:
            continue # Skip if there's no stock data
        start_price = start_price.iloc[0] # Take the first trading day
        # on or after that target date

        target_date = date + pd.Timedelta(days=days_after) # Compute
        # the calendar date x days later
        end_price = stock_data[stock_data.index >= target_date] #
        # Filter to the first trading day on/after that target date
```

### AI-Generated Version

```
def compute_returns(spike_dates, stock_data,
                    days_after): returns = []
    for date in spike_dates:
        try:
            start_price = stock_data.loc[stock_data.index >=
            date].iloc[0] target_date = date +
            pd.Timedelta(days=days_after)
            end_price = stock_data.loc[stock_data.index >=
            target_date].iloc[0]
            returns.append(percent_change(start_price, end_price))
        except IndexError:
            continue
    return returns
```

The original code is easier to follow because it separates each step of the process, whereas the AI-generated version is more compact and uses different logic (try/except) to handle the missing values. Both approaches function the same: locate the starting price, identify the target date, find the ending price, and lastly, compute the percentage return.

AI-generated code offers advantages in improving flow and reducing redundancies, but typically human-written code is easier to follow and debug, making it much more user-friendly.

## 5. Testing Methodology

Testing focused on the main calculation functions and used simple assertion-based tests to verify that the code behaved correctly. Synthetic data was created as a control input for validating these tests. The methodology is limited in scope and would likely not be sufficient for more demanding stress testing.

## **6. Analysis Results**

The analysis revealed a pattern across many tickers: returns often increased during the first two days after a spike, then dipped noticeably on the third day, and then tended to recover over the following days. Although this pattern requires deeper statistical validation before asserting any conclusions, it was a consistent observation in the project.

## **7. Reflection**

The program successfully answered the research question of how stocks behave after VIX spikes. The results revealed an interesting pattern that could be of interest to market analysts and traders, although this project was primarily an academic project rather than an investment tool. Throughout the course of this project, many errors and hurdles were encountered, some more manageable than others, but ultimately the finished code served as evidence of progress in writing, testing, reading, and organizing code.